

The exam is open books, notes, printed materials. You are allowed to browse the web. Consulting/discussing with anyone which includes asking questions in any forum during the exam is strictly prohibited. *Unless specified explicitly, assume 64-bit X86 architecture with multiple CPUs.* Be precise with your answers. No marks for vague answers. Make reasonable assumptions and state them explicitly. *All the best!*

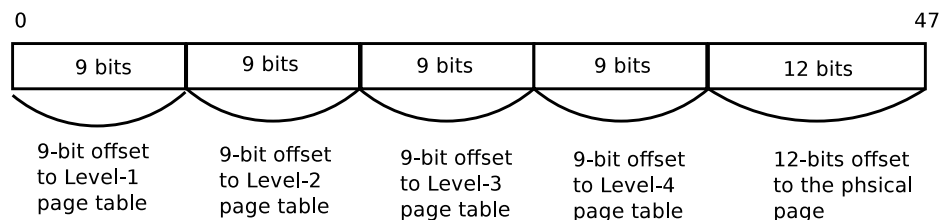
1. In the classical `fork()` implementation, the child inherits the parent address space. This requires creating duplicate content in memory at the time of `fork()` which can be inefficient, both in terms of memory and CPU usage. One possible optimization of the `fork()` system call (without changing its semantics) is to share the physical memory between the parent and the child processes as long as possible. A possible strategy could be to perform the copy of the memory content *only when* either the parent or the child process performs a *write* to the corresponding address. Let us call this optimized version `forkopt()`. Answer the following questions assuming a virtual memory system with paging,

$$3 + 10 + 2 = 15$$

- Compare the efficiency of launching a new executable using `(forkopt + exec)` vs. `(fork + exec)`.
 - Describe a strategy for implementation of `forkopt`. Specifically, explain how the address space and the page table maintenance will change along with description of changes to different OS subsystems using pseudocode. Clearly mention the changes to existing OS structures and procedures.
 - Between `fork` and `forkopt`, which will result in more number of page faults and why?
2. One of the challenges in real system performance analysis is to zero down to the software bottlenecks at the code level. In practical applications built from a large code base, isolating performance pain points is a big challenge. You are assigned with the task to design *an application transparent mechanism* in the OS to find K user-level functions consuming most of the CPU time for any single threaded process. Specifically, you are required to provide two system calls allowed to be invoked from any user process with root user privileges as explained below,
- `int analyze(int pid, int k)`: Start capturing the execution behavior of the process identified by `pid` to find the k most time consuming functions. Assume that, there is an existing capability in the OS to find out the function name given an instruction address for any process. You are required to explain the core logic; no need to explain the validation and error handling.
 - `int dump(int pid, char *buf)`: Copy the collected statistics (name of top k functions sorted based on their CPU consumption in descending order) for the process identified by `pid` into the `buf` argument. Assume that, the `buf` is of sufficient length. This system call also stops collecting statistics for the process.

Give the pseudocode along with precise explanation to implement the above functionality. If you are introducing new OS structures or changing any existing structures, explain them clearly. While your design can slow down the process under analysis, you should try to minimize the impact on other processes in the system. Comment about the accuracy and performance overheads of the proposed system. Note that, you are not allowed to modify the executable in any manner e.g., introducing `int 3` etc. Further, do not assume any special hardware assistance which is not covered during the course.

20



3. Consider a 4-level paging system for forty-eight bit (48-bit) virtual address (as shown above) and a page size of four kilobytes (4KB). Page table entries at each level of the page table is 8 bytes. Answer the following questions,

$$2 + 3 + 2 + 3 + 5 = 15$$

- At a given time, three valid entries were present in the Level-1 page table of a given process P_1 . The OS wants to allocate new virtual address requests in such a manner that a new entry in Level-1 is avoided. What is the maximum virtual memory usage of the process P_1 after which the OS will be forced to allocate a new entry in Level-1 page table?
- If a process successfully unmaps 2GB memory from an existing mapping, what is the maximum and minimum amount of free memory increase in the system? (ignore the memory usage change due to manipulation of VM related data structure like `vm_area` etc.)

- (c) The maximum physical memory supported by the above paging system is given as 512 petabyte (512 PB, where $1 \text{ PB} = 2^{50}$ bytes). If five bits are used to maintain the access flags, how many bits remain unused in any page table entry?
- (d) A user process in the system has allocated 1 gigabyte (1 GB) of virtual memory where each virtual address is mapped to distinct physical address. Find out the lower bound on memory usage to maintain the page table for this process.
- (e) For the previous part of the question, find the upper bound on memory usage for maintaining the page tables.

4. Consider the following C program to maintain a sorted list of non-negative integers.

10

```

1  struct node{
2      unsigned item;
3      struct node *next;
4      Semaphore s;
5  };
6
7  struct node head = {0, NULL, SEM_INIT(1)}; // Semaphore initialized to 1
8
9  int insert_sorted(unsigned element)
10 {
11     struct node *tmp = &head, *newnode;
12     while(tmp->next && tmp->next->item < element)
13         tmp = tmp->next;
14     newnode = malloc(sizeof(struct node));
15     if(!newnode)
16         return -1;
17     newnode->item = element;
18     newnode->s = SEM_INIT(1); //Initialize semaphore for new node
19     newnode->next = tmp->next;
20     tmp->next = newnode;
21     return 0;
22 }
```

You have a requirement to enhance the sorted list insertion functionality that can handle concurrent insertions using a per-node binary semaphore. You *do not want* to lock the head of the list for the entire duration of insertion which will serialize the insertion functionality. Design appropriate synchronization logic (wherever required) to allow concurrent insertion ensuring correctness. Assume that there are no other concurrent operations on the sorted list and the pointer update at Line#19 is atomic. You are required to provide the code for the modified version of `insert_sorted`.

5. Consider an indexed file system found in the UNIX operating system. The inodes have an indexed allocation scheme as follows: eight direct block address, two single-indirect block address and eight double indirect block addresses. Assume that, the block size and on-disk inode size are four kilobytes (4KB) and 256 bytes, respectively. Block address in the system is 4 bytes long. Further, assume that every directory maintains an array of flat directory entries having the following structure,

```

struct dentry{
    u32 inode-number;
    char name[252];
}
```

Assume that, in all scenarios the operations are successful and no other disk blocks are cached in memory apart from the cached inodes. Further, assume that the disk I/O operations are performed in units of block size i.e., 4KB. Answer the following questions.

$2 + 4 + 4 + 5 = 15$

- (a) You have written a program to create as many files as possible under an initially empty directory. How many files your program can successfully create?
- (b) What is the minimum and maximum number of disk block accesses required for creating a new file in any existing directory?
- (c) A user process successfully reads one megabyte (1MB) from a random offset of a 1GB file. Assuming there is no caching of disk blocks and the file inode is already present in memory, what is the minimum and maximum amount of disk read required to serve the read request?

- (d) A user process opens and reads an existing file at `/home/user/courses/os/notes.txt` of size four kilobytes, where `/` is the file system mount point. Assume that, all the inodes required to access the file are present in memory. What is the maximum and minimum number of disk block accesses required to complete the read operation?